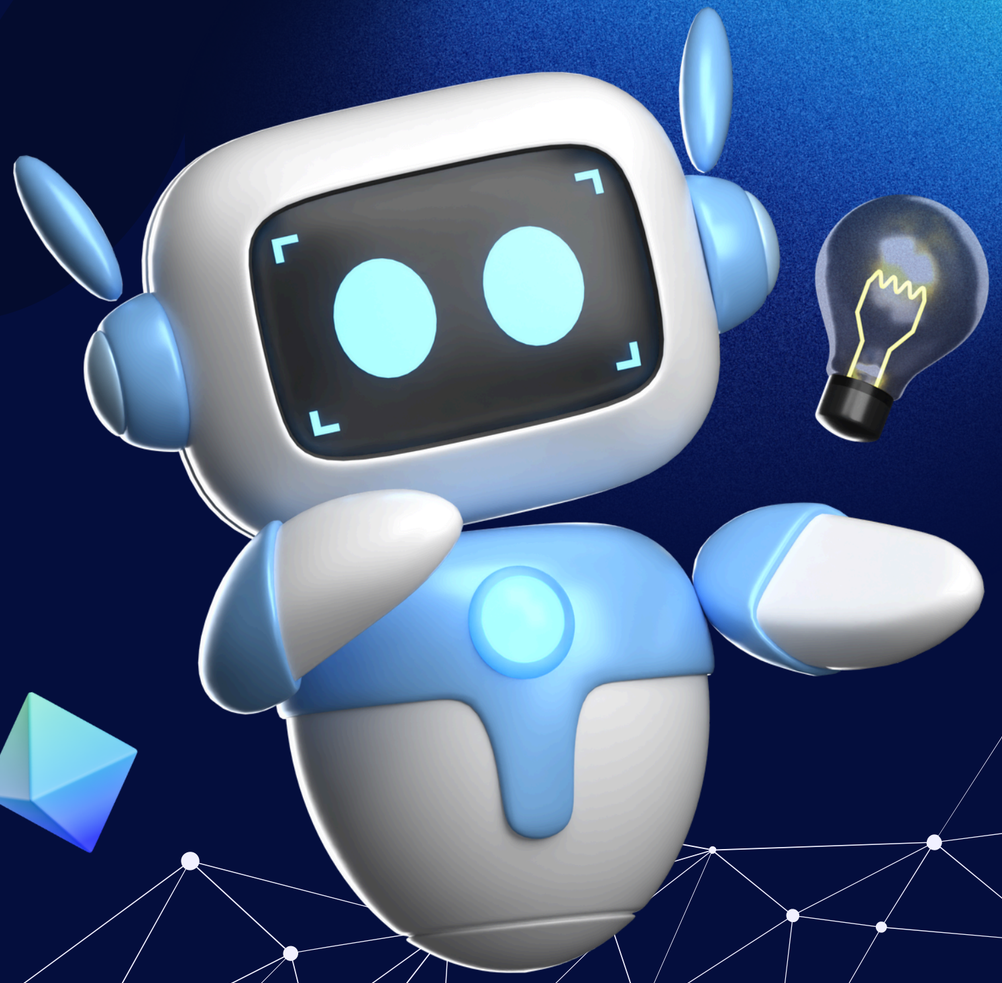
MINI LANGUAGE MODEL HARDWARE ACCELERATOR IN VHDL

PC/CP319 – AI MODEL DESIGN STREAM,
MILESTONE 2

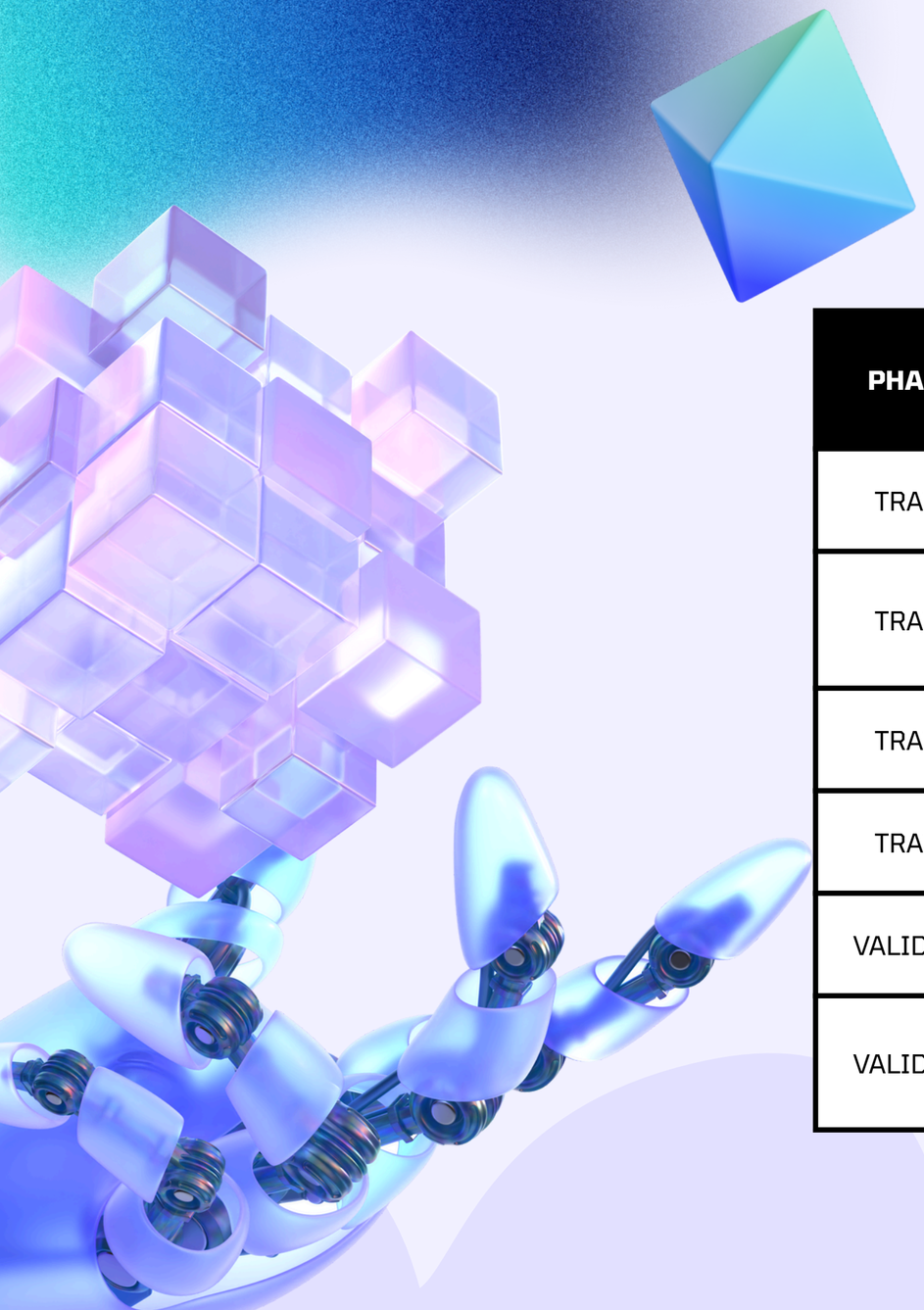


By:
Ahmed Elag & Marc Niven

BIG PICTURE: PROJECT OVERVIEW

- Implement a very small 2-layer neural network in hardware.
- Inputs are short sentences encoded as 8-feature vectors.
- Outputs are 4 reply categories (e.g. “Hi there!”, “I’m fine.”, “I’m AI.”, “See you soon.”).
- Hardware supports a training phase and a prediction/validation phase.

What are we trying to achieve?

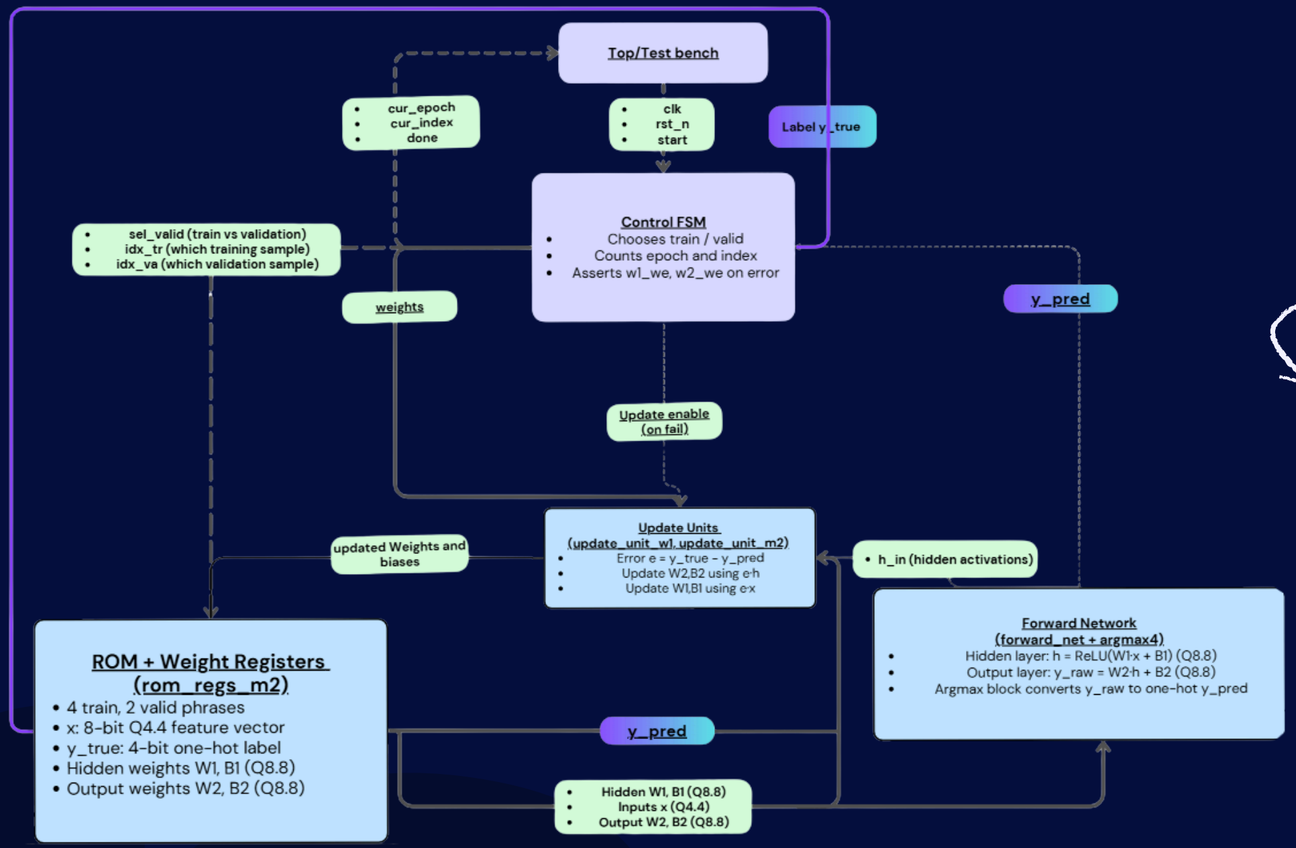


SENTENCE ENCODINGS AND TARGETS

PHASE	INDEX	INPUT SENTENCE	8-BIT INPUT ENCODING	TARGET REPLY	REPLY ENCODING
TRAIN	0	HELLO	10000000	HI THERE	0001
TRAIN	1	I'M STRESSED	01000000	TAKE A SHORT BREAK	0010
TRAIN	2	I'M TIRED	00100000	GET SOME REST	0100
TRAIN	3	I'M GOING TO SLEEP	00010000	GOOD NIGHT	1000
VALIDATE	0	HELLO + IM STRESSED	11000000	TAKE A SHORT BREAK	0001
VALIDATE	1	IM TIRED + I WILL GO SLEEP	00110000	GOODNIGHT	0100

INPUTS USE Q4.4 FIXED-POINT, ONE-HOT STYLE; TARGETS ARE 4-BIT ONE-HOT REPLY CLASSES

HARDWARE ARCHITECTURE



ROM+REG

- Tiny hardware “language model” built as a 2-layer neural network in VHDL.
- Encodes short sentences as 8-feature vectors and maps them to 4 reply classes.
- Supports a training phase to learn weights and a prediction/validation phase to test them.

Forward Net

- Stores training and validation sentence vectors plus their one-hot labels.
- Holds trainable hidden-layer (W1/B1) and output-layer (W2/B2) weights.
- Feeds the correct data into the network and writes back updated W2/B2 after learning steps.

UPDATE UNIT

- Computes h=**ReLU**(W1x+B1) and y= W2h + B2 in fixed-point.
- Uses ReLU in the hidden layer and argmax on the outputs to produce a one-hot reply.
- Same hardware is reused for both training and validation forward passes.

CONTROL FSM

- Compares true labels and predictions to generate a simple error for each output neuron.
- Updates both hidden and output weights and biases using a small, shift-based learning rate.
- Shows clear improvement over epochs while keeping the learning rule hardware-friendly.

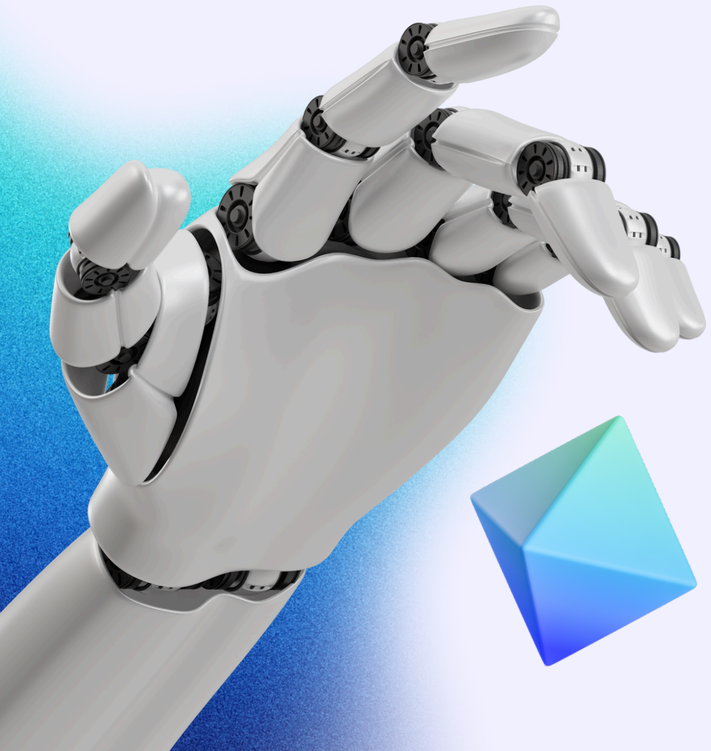
CONTROL FSM: TRAINING AND VALIDATION

- LOOPS OVER 4 TRAINING SAMPLES FOR EACH EPOCH.
- TRACKS MISCLASSIFICATIONS; STOPS EARLY IF NO ERRORS OR HITS MAX EPOCH.
- THEN SWITCHES TO 2 VALIDATION SAMPLES.

NUMERICAL MODEL AND FIXED-POINT

EQUATIONS

- $h = \mathrm{ReLU}(W_1 x + B_1)$
- $y = W_2 h + B_2$
- $\hat{c} = \arg\max_k y_k$ (predicted class)



- Inputs: Q4.4, 8-bit signed (1.0 = 16).
- Hidden & outputs: Q8.8, 16-bit signed (1.0 = 256).
- Both W1, B1 and W2, B2 are trainable.
- Update: perceptron-style step with small learning rate (shift-based).

x → W1/B1 → ReLU → W2/B2 → argmax → one-hot reply

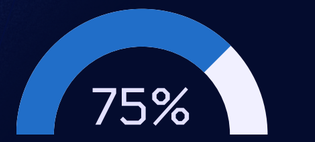
TRAINING BEHAVIOUR

EPOCH 0

MOSTLY FAILS

EPOCH 0 - TRAINING

Idx	Input text	y_true(bits/text)	y_pred(bits/text)	Result
0	Hello	0001 Hey there!	0001 Hey there!	PASS
1	I'm stressed	0010 Take a short br	0001 Hey there!	FAIL
2	I'm tired	0100 Get some rest.	0010 Take a short br	FAIL
3	I'm going to sleep	1000 Good night!	0100 Get some rest.	FAIL



EPOCH 1

MIXED PASS/FAIL RATIO

EPOCH 1 - TRAINING

Idx	Input text	y_true(bits/text)	y_pred(bits/text)	Result
0	Hello	0001 Hey there!	1000 Good night!	FAIL
1	I'm stressed	0010 Take a short br	0010 Take a short br	PASS
2	I'm tired	0100 Get some rest.	0100 Get some rest.	PASS
3	I'm going to sleep	1000 Good night!	1000 Good night!	PASS

EPOCH 2

ALL PASS = CONVERGENCE

EPOCH 2 - TRAINING

Idx	Input text	y_true(bits/text)	y_pred(bits/text)	Result
0	Hello	0001 Hey there!	0001 Hey there!	PASS
1	I'm stressed	0010 Take a short br	0010 Take a short br	PASS
2	I'm tired	0100 Get some rest.	0100 Get some rest.	PASS
3	I'm going to sleep	1000 Good night!	1000 Good night!	PASS

LEARNED WEIGHTS

HIDDEN LAYER
W1, B1 – Q8.8

- **Q8.8: value / 256**
- **224 ≈ 0.88**
- **288 ≈ 1.13**

```
=== W1/B1 UPDATE @ time 275 ns
B1[0..3] = [-0.125, 0.125, 0.125, 0.125]
W1 (4x4 block):
  row 0 = [0.875, 0.000, 0.000, 0.000]
  row 1 = [0.000, 1.125, 0.000, 0.000]
  row 2 = [0.000, 0.000, 1.125, 0.000]
  row 3 = [0.000, 0.000, 0.000, 1.125]
```

- First 4 hidden neurons still act like “copies” of the 4 input bits (word flags).
- Diagonal weights grew slightly above 1 or just below 1, so each neuron is tuned to its own word.
- Small positive / negative biases shift when each hidden neuron turns on.

OUTPUT LAYER
W2, B2 – Q8.8

- **64 ≈ 0.25**
- **56 ≈ 0.22**
- **56 ≈ -0.22**
- **-64 ≈ -0.25**

- **W2** (**rows = reply classes**
- **cols = hidden 0..7**)

```
=== W2/B2 UPDATE @ time 275 ns
B2 = [0.000, 0.000, 0.000, 0.000]
W2 (4x4 block):
  row 0 = [0.250, -0.219, 0.031, 0.031]
  row 1 = [0.000, 0.219, -0.250, 0.000]
  row 2 = [0.000, 0.000, 0.219, -0.250]
  row 3 = [-0.250, 0.000, 0.000, 0.219]
```

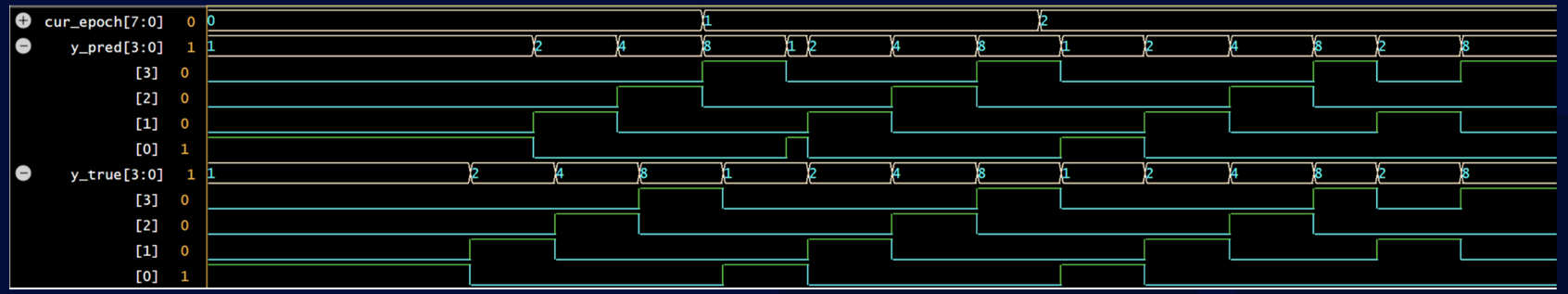
- Each reply class mostly listens to its own “word neuron” and often subtracts competing words.
  - example 1: “Hi there!”: strong + on hidden 0, negative on hidden 1.
  - example 2: “See you soon.”: strong + on hidden 3, negative on hidden 0.
- This creates a simple competition: the matching class gets the highest score and wins argmax.

VALIDATION ON MIXED SENTENCES

VALIDATION AFTER EPOCH 2

```
================================================================================
VALIDATION AFTER EPOCH 2
================================================================================
Idx  Input text                 y_true(bits/text)     y_pred(bits/text)     Result
--------------------------------------------------------------------------------
0    Hello + I'm stressed       0010 Take a short br  0010 Take a short br  PASS
1    Tired + Going to sleep     1000 Good night!      1000 Good night!      PASS
```

WAVEFORM:



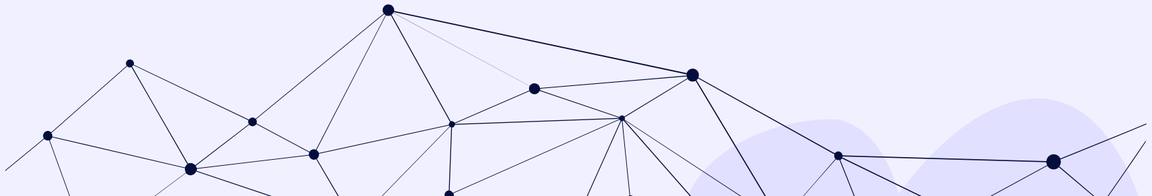
CONCLUSION AND FUTURE WORK

CONCLUSION

- Implemented a 2-layer mini-LLM in VHDL with training and validation modes.
- Demonstrated convergence on a small sentence–reply dataset.
- Used ModelSim to visualise FSM, learning, and validation.

FUTURE WORK AND IMPROVEMENTS

- Increase dataset size and number of outputs.
- Explore different activation functions or learning rates.



THANK YOU